CO2 ASSESSSMENT TOOL 2

CSA1305-TOC

V.Navyasree(192373050)

1. Design the Regular Expression for the Language Represented by the Given Automaton

Given Automaton

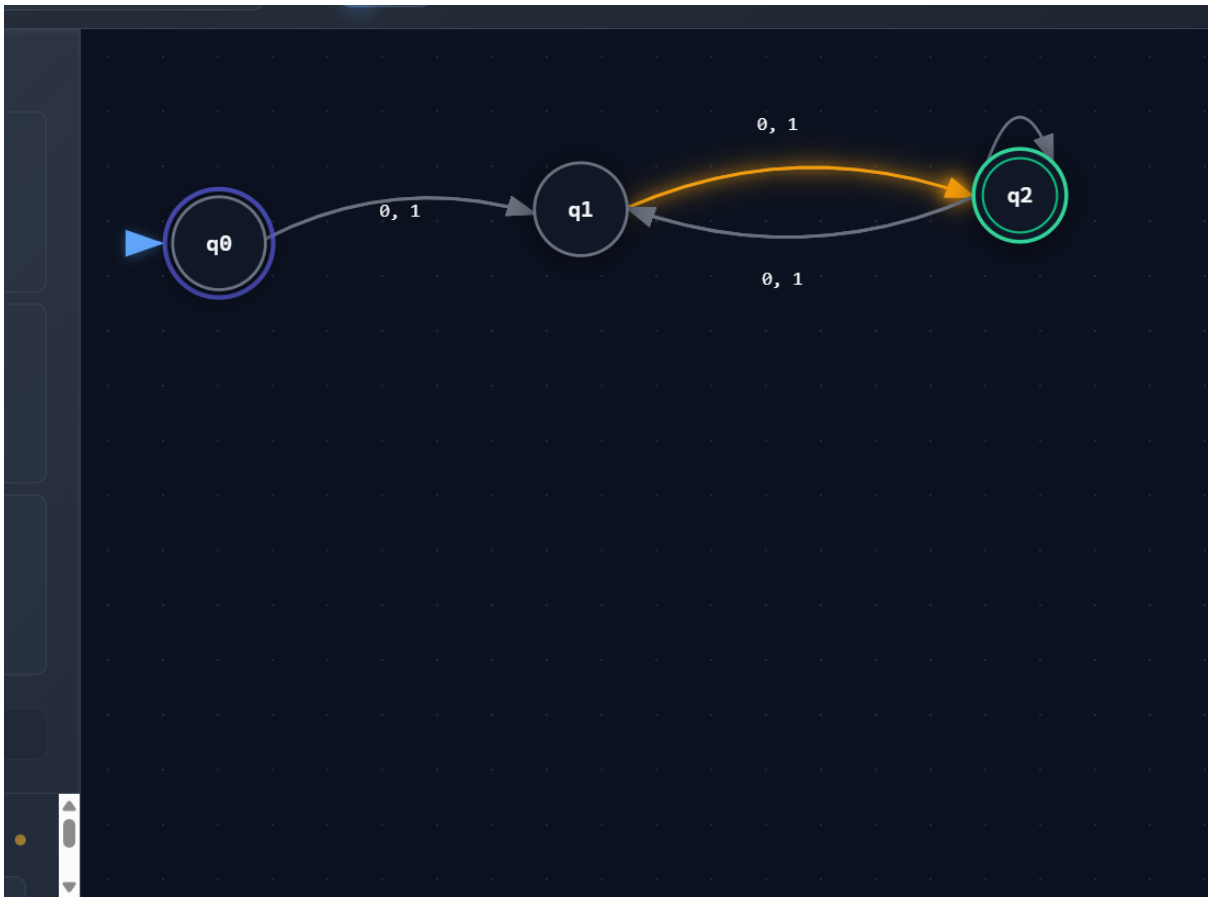
States:

- q0 → Start State
- q1 → Intermediate State
- q2 → Final State

Transitions:

- q0 → q1 on **0,1**
- q1 → q2 on **0,1**
- q2 → q1 on **0,1**

Step 1: Analyze the DFA



To reach the final state:

- One symbol moves from q0 to q1.
- Another symbol moves from q1 to q2.

After reaching q2, the automaton alternates between q2 and q1 on every input symbol.

Hence accepted strings have **even length (minimum length = 2).**

Examples Accepted:

00

01

10

11

0011

1100

1010

111100

Rejected:

0

1

000

111

101

Regular Expression

Since every symbol may be either 0 or 1,

the regular expression is

or equivalently,

Applications

- Binary protocol validation
- Digital communication

- Network packet verification
- Lexical analysis

Conclusion

The automaton accepts all binary strings having **even length**, represented by

2. Grammar

Given

$S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$T \rightarrow bTc \mid \varepsilon$

$U \rightarrow aUb \mid \varepsilon$

(A) Language Defined

Production 1

$S \rightarrow aSc$

Generates

$a^n c^n \backslash$

Production 2

$T \rightarrow bTc$

Generates

$b^n c^n$

Production 3

$U \rightarrow aUb$

Generates

$a^n b^n$

Hence,

(B) Derivation Tree for aaabbc

The string

aaabbc

cannot be produced.

Reason:

The grammar produces

$a^n c^n$

or

$b^n c^n$

or

$a^n b^n$

but never

aaa bb c

Therefore

No derivation tree exists.

(C) Ambiguity

Definition:

A grammar is ambiguous if one string has more than one parse tree.

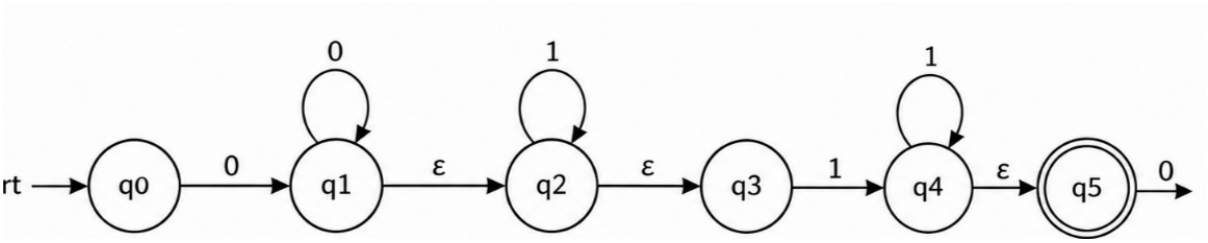
Here every production generates a different language.

No string has two different parse trees.

Therefore

3. Design an ε-NFA for

00*1*11*0



Regular Expression

0 0* 1* 1 1* 0

Simplified

0+ 1+ 0

ε-NFA Construction

States

q0 → q1 → q2 → q3 → q4

Transitions

q0 --0--> q1

q1 --0--> q1

q1 --ε--> q2

q2 --1--> q2

q2 --1--> q3

q3 --1--> q3

q3 --0--> q4

Final State

q4

Examples

Accepted

010

0010

0001110

0011110

Rejected

100

1110

000

Applications

- Compiler lexical analyzer
- Pattern matching
- Intrusion detection

4(i) Show that if L_1 and L_2 are Regular then (L_1+L_2) is Regular

Closure Property

Regular languages are closed under Union.

Suppose

L_1

is recognized by DFA M_1

and

L_2

is recognized by DFA M_2.

Construct the product DFA

$Q = Q_1 \times Q_2$

Accept if either DFA accepts.

Therefore

Applications

- Spam filtering
- Firewalls
- Search engines
- Compiler scanners

4(ii) Is

Regular?

Answer

No.

Pumping Lemma Proof

Assume

L

is Regular.

Take

w=a^q

where

q

is prime and larger than pumping length.

Split

w=xyz

where

|xy|≤p

Hence

y=a^k

Pump

i=2

Length becomes

q+k

(or by choosing an appropriate pumping count, a composite length is obtained), which is not guaranteed to remain prime.

Contradiction.

Therefore

5. DFA Minimization

Step 1

Separate

Final

{q1,q2,q4}

Non-final

{q0,q3,q5}

Step 2

Compare transitions.

Equivalent states are merged if every transition leads to equivalent states.

For the given DFA, after partition refinement the equivalent pair is:

(All other states remain distinguishable.)

Step 3

Merge

q1

and

q4

into one state.

Minimized States

q0

q3

(q1,q4)

q2

q5

Advantages

- Reduced memory
- Lower hardware cost
- Faster execution
- Less power consumption
- Efficient embedded implementation

Applications

- ATM machines
- Smart cards
- IoT devices
- Embedded controllers
- Digital communication systems

Conclusion

The minimized DFA recognizes the **same language** as the original DFA but with fewer states, resulting in simpler hardware and faster execution.